

CRRSS Usage Guide

Version 1.0.0 | Phase 1B Stage 4: Build System Integration & Documentation

Comprehensive guide for building, installing, and using the CRRSS tooling system.

Table of Contents

- [1. Overview](#)
 - [2. Installation](#)
 - [3. Building CRRSS](#)
 - [4. Command-Line Tools](#)
 - [5. Component Usage](#)
 - [6. Configuration](#)
 - [7. Integration with Build System](#)
 - [8. Testing](#)
 - [9. Examples](#)
 - [10. Troubleshooting](#)
-

Overview

CRRSS (Code Review, Reliability, and Static Safety System) is a comprehensive tooling suite for kernel development providing:

- **BPME** (Bug Prior Mapping Engine): Pattern-based bug detection and prediction
- **SCIV** (Self-Check Internal Validator): Code quality and standards validation
- **Memory Integration Layer**: Memory subsystem analysis and leak detection
- **MSM** (Memory-Safety Maniac Profile): Comprehensive memory safety analysis

Key Features

- ✓ Automated bug detection based on historical patterns
 - ✓ Real-time memory safety analysis
 - ✓ Code quality validation
 - ✓ Comprehensive reporting (text, JSON, HTML)
 - ✓ Integration with BDI build system
 - ✓ Optional pre-commit hooks
 - ✓ Command-line and API interfaces
-

Installation

Prerequisites

- **GCC** with C23 support (GCC 13+) or **Clang**

- **Make** build system
- **pthread**s library
- **libm** (math library)
- **CMake** 3.16+ (optional, for CMake builds)

Quick Install (Recommended)

From the BDI repository root:

```
# Build CRRSS
make crrss

# Run tests
make crrss-test

# Install to system (optional)
sudo make crrss-install
```

Manual Installation

```
# Navigate to CRRSS directory
cd tools/crrss

# Build everything
make all

# Run tests
make test

# Install (default: /usr/local)
sudo make install

# Or install to custom location
make install INSTALL_PREFIX=$HOME/.local
```

CMake Build (Alternative)

```
cd tools/crrss
mkdir build && cd build
cmake ..
make
make test
sudo make install
```

Building CRRSS

Build Modes

Release Build (Default)

```
make all
# or
make BUILD_MODE=release all
```

- Maximum optimization (O3, LTO)
- Native architecture targeting
- No debug symbols

Debug Build

```
make BUILD_MODE=debug all
```

- No optimization (O0)
- Full debug symbols (g3)
- AddressSanitizer and UndefinedBehaviorSanitizer
- Additional assertions

Build Targets

```
make all           # Build library, tool, and tests
make lib           # Build libcrrss.a only
make tool          # Build crrss command-line tool only
make tests         # Build test suite only
make clean         # Clean build artifacts
make validate      # Validate build output
```

Build Configuration

```
# Custom compiler
make CC=clang all

# Custom install prefix
make install INSTALL_PREFIX=/opt/crrss

# Parallel build
make -j$(nproc) all
```

Command-Line Tools

Main CRRSS Tool

The `crrss` command-line tool provides access to all CRRSS functionality.

General Syntax

```
crrss <command> [options]
```

Global Options

- `--version` : Show version information
 - `--help` : Show help message
 - `--verbose` : Enable verbose output
 - `--quiet` : Suppress non-error messages
-

MSM Command (Memory-Safety Maniac)

Analyze files or directories for memory safety issues.

Basic Usage

```
# Analyze single file
crrss msm -f <file>

# Analyze directory
crrss msm -d <directory>

# Generate report
crrss msm -d <directory> --report <output> --format <format>
```

Options

- `-f, --file <path>` : Analyze single file
- `-d, --directory <path>` : Analyze directory (recursive)
- `--report <path>` : Output report to file
- `--format <fmt>` : Report format (text, json, html)
- `--max-issues <num>` : Maximum issues to report
- `--severity <level>` : Minimum severity (info, warning, error, critical)
- `--category <cat>` : Filter by category (memory, pointer, buffer, etc.)

Examples

```
# Analyze kernel memory module
crrss msm -f moduler_kernel/memory/memory.c

# Analyze entire kernel with HTML report
crrss msm -d moduler_kernel/ --report msm_report.html --format html

# Show only critical issues
crrss msm -d moduler_kernel/ --severity critical

# Filter memory leak issues
crrss msm -d moduler_kernel/ --category memory-leak
```

Query Command

Query bug predictions and historical patterns.

Basic Usage

```
crss query [options]
```

Options

- `-p, --priority <level>` : Filter by priority (P0, P1, P2, P3)
- `-c, --category <cat>` : Filter by category (memory, concurrency, logic, etc.)
- `-f, --file <path>` : Query specific file
- `-d, --details` : Show detailed information
- `-n, --max-results <num>` : Maximum results to display

Examples

```
# Query all P0 (critical) issues
crss query --priority P0 --details

# Query memory-related issues
crss query --category memory

# Query issues in specific file
crss query --file moduler_kernel/memory.c --details

# Query with combined filters
crss query --priority P1 --category concurrency --max-results 20
```

Stats Command

Display codebase statistics and analysis results.

Basic Usage

```
crss stats [options]
```

Options

- `-d, --directory <path>` : Analyze directory
- `-m, --memory` : Show memory statistics
- `-v, --validation` : Show validation statistics
- `--format <fmt>` : Output format (text, json, csv)
- `--export <path>` : Export statistics to file

Examples

```
# Show all statistics
crrss stats

# Show memory statistics for directory
crrss stats --directory moduler_kernel/ --memory

# Export statistics as JSON
crrss stats --format json --export stats.json

# Show validation results
crrss stats --validation
```

Component Usage

BPME (Bug Prior Mapping Engine)

Initialization

```
#include "bpme/bpme.h"

bpme_config_t config = {
    .enable_pattern_matching = true,
    .confidence_threshold = 0.7,
    .max_predictions = 100
};

bpme_context_t* ctx = bpme_initialize(&config);
```

Analyze File

```
bug_prediction_t predictions[100];
uint32_t num_predictions = 0;

crrss_status_t status = bpme_analyze_file(
    ctx,
    "memory.c",
    predictions,
    100,
    &num_predictions
);

if (status == CRRSS_SUCCESS) {
    for (uint32_t i = 0; i < num_predictions; i++) {
        printf("Bug: %s at line %u (confidence: %.2f)\n",
            predictions[i].description,
            predictions[i].line_number,
            predictions[i].confidence);
    }
}
```

Analyze Directory

```
crss_status_t status = bpme_analyze_directory(
    ctx,
    "moduler_kernel/",
    predictions,
    100,
    &num_predictions,
    true // recursive
);
```

Query by Priority

```
crss_status_t status = bpme_query_by_priority(
    ctx,
    BUG_PRIORITY_P0, // Critical issues only
    predictions,
    100,
    &num_predictions
);
```

Cleanup

```
bpme_shutdown(ctx);
```

SCIV (Self-Check Internal Validator)

Initialization

```
#include "sciv/sciv.h"

sciv_config_t config = {
    .enable_strict_mode = true,
    .enable_style_checks = true,
    .max_function_complexity = 15,
    .coding_standard = "kernel"
};

sciv_context_t* ctx = sciv_initialize(&config);
```

Validate File

```
validation_issue_t issues[100];
uint32_t num_issues = 0;

crrss_status_t status = sciv_validate_file(
    ctx,
    "driver.c",
    issues,
    100,
    &num_issues
);

if (status == CRRSS_SUCCESS) {
    for (uint32_t i = 0; i < num_issues; i++) {
        printf("Issue: %s at line %u (severity: %d)\n",
            issues[i].message,
            issues[i].line_number,
            issues[i].severity);
    }
}
```

Calculate Complexity

```
uint32_t complexity = 0;
crrss_status_t status = sciv_calculate_complexity(
    ctx,
    "scheduler.c",
    "schedule", // function name
    &complexity
);

printf("Function complexity: %u\n", complexity);
```

Get Quality Score

```
double quality_score = 0.0;
crrss_status_t status = sciv_get_quality_score(
    ctx,
    "memory.c",
    &quality_score
);

printf("Code quality score: %.2f/100\n", quality_score);
```

Cleanup

```
sciv_shutdown(ctx);
```

Memory Integration Layer

Initialization

```
#include "memory_layer/memory_integration.h"

memory_integration_config_t config = {
    .enable_leak_detection = true,
    .enable_use_after_free_detection = true,
    .track_allocations = true,
    .max_tracked_allocations = 10000
};

memory_integration_context_t* ctx = memory_integration_initialize(&config);
```

Detect Memory Leaks

```
leak_detection_report_t report = {0};
report.max_records = 100;

crrss_status_t status = memory_integration_detect_leaks(ctx, &report);

if (status == CRRSS_SUCCESS) {
    printf("Potential leaks: %u\n", report.potential_leaks);
    printf("Total leaked: %lu bytes\n", report.total_leaked_bytes);

    for (uint32_t i = 0; i < report.num_records; i++) {
        printf("    Leak: %lu bytes at %s:%u\n",
            report.records[i].size,
            report.records[i].file,
            report.records[i].line);
    }
}
```

Track Allocation

```
void* ptr = malloc(1024);
memory_integration_track_allocation(ctx, ptr, 1024, "memory.c", 42);
```

Track Deallocation

```
memory_integration_track_deallocation(ctx, ptr);
free(ptr);
```

Cleanup

```
memory_integration_shutdown(ctx);
```

MSM (Memory-Safety Maniac Profile)

Initialization

```
#include "msm/msm.h"

msm_config_t config = {
    .enable_runtime_tracking = true,
    .enable_static_analysis = true,
    .enable_stack_traces = true,
    .max_tracked_allocations = 10000,
    .stack_trace_depth = 32
};

msm_context_t* ctx = msm_initialize(&config);
```

Analyze File

```
msm_analysis_result_t result = {0};

crrss_status_t status = msm_analyze_file(ctx, "memory.c", &result);

if (status == CRRSS_SUCCESS) {
    printf("Total issues: %u\n", result.total_issues);
    printf("  Memory leaks: %u\n", result.memory_leaks);
    printf("  Use-after-free: %u\n", result.use_after_free);
    printf("  Double-free: %u\n", result.double_free);
    printf("  NULL dereference: %u\n", result.null_dereference);
    printf("  Buffer overflow: %u\n", result.buffer_overflow);
}
```

Analyze Directory

```
msm_analysis_result_t result = {0};

crrss_status_t status = msm_analyze_directory(
    ctx,
    "moduler_kernel/",
    &result,
    true // recursive
);
```

Generate Report

```
msm_report_config_t report_config = {
    .format = MSM_REPORT_FORMAT_HTML,
    .include_stack_traces = true,
    .include_code_context = true,
    .severity_threshold = MSM_SEVERITY_WARNING
};

crrss_status_t status = msm_generate_report(
    ctx,
    &result,
    "msm_report.html",
    &report_config
);
```

Cleanup

```
msm_shutdown(ctx);
```

Configuration

Environment Variables

Configure CRRSS behavior with environment variables:

```
# Enable/disable components
export CRRSS_ENABLE_BPME=1
export CRRSS_ENABLE_SCIV=1
export CRRSS_ENABLE_MSM=1

# Verbosity
export CRRSS_VERBOSE=1
export CRRSS_DEBUG=0

# Performance
export CRRSS_MAX_THREADS=8
export CRRSS_CACHE_SIZE=1000

# Pre-commit hook configuration
export CRRSS_ENABLED=1
export CRRSS_STRICT=0
export CRRSS_MAX_ISSUES=10
```

Configuration Files

CRRSS looks for configuration in:

1. `$HOME/.crrss/config`
2. `$PWD/.crrss`
3. Command-line options (highest priority)

Example configuration file (`.crrss`):

```
[global]
verbose = true
max_threads = 4

[bpme]
enable = true
confidence_threshold = 0.7
max_predictions = 100

[sciv]
enable = true
strict_mode = true
max_complexity = 15

[msm]
enable = true
runtime_tracking = true
static_analysis = true
stack_traces = true
```

Integration with Build System

Makefile Integration

From BDI repository root:

```
# Build CRRSS
make crrss

# Run CRRSS tests
make crrss-test

# Analyze BDI codebase
make crrss-check

# Analyze specific file
make crrss-analyze FILE=moduler_kernel/memory.c

# Install CRRSS
sudo make crrss-install

# Clean CRRSS
make crrss-clean
```

From CRRSS Directory

```
cd tools/crrss

# Build all
make all

# Build library only
make lib

# Build tool only
make tool

# Build tests only
make tests

# Run tests
make test

# Analyze BDI codebase
make check-codebase

# Analyze specific file
make analyze-file FILE=../../moduler_kernel/memory.c
```

Pre-commit Hook Integration

```
# Install pre-commit hook
./git-hooks/install-hooks.sh install

# Check status
./git-hooks/install-hooks.sh status

# Test hook
./git-hooks/install-hooks.sh test

# Uninstall
./git-hooks/install-hooks.sh uninstall
```

See `.git-hooks/README.md` for detailed pre-commit hook documentation.

Testing

Run All Tests

```
# From BDI root
make crrss-test

# From CRRSS directory
make test
```

Run Individual Tests

```
# From CRRSS directory
make test-bpme      # BPME tests
make test-sciv      # SCIV tests
make test-memory    # Memory Integration tests
make test-msm       # MSM tests
```

Run Tests Directly

```
cd tools/crrss
./build/test/test_bpme
./build/test/test_sciv
./build/test/test_memory
./build/test/test_msm
```

Verbose Test Output

```
make test VERBOSE=1
```

Examples

Example 1: Analyze Kernel Module

```
# Analyze memory module
crrss msm -d moduler_kernel/memory/ --report memory_analysis.txt --format text

# Check results
cat memory_analysis.txt
```

Example 2: Find Critical Issues

```
# Query all P0 issues
crrss query --priority P0 --details > critical_issues.txt

# Review
less critical_issues.txt
```

Example 3: Validate Code Quality

```
# Run validation on scheduler
crrss stats --directory moduler_kernel/scheduler/ --validation

# Check quality score
crrss stats --directory moduler_kernel/scheduler/ --format json | jq '.quality_score'
```

Example 4: Memory Leak Detection

```
# Full memory analysis
crrss msm -d moduler_kernel/ --category memory-leak --report leaks.html --format html

# Open in browser
firefox leaks.html
```

Example 5: Pre-commit Analysis

```
# Analyze staged files before commit
git add moduler_kernel/memory.c
git commit -m "Fix memory leak"
# Hook runs automatically

# Or manual analysis
crrss msm -f moduler_kernel/memory.c
```

Example 6: API Usage

```
#include "msm/msm.h"

int main() {
    // Initialize MSM
    msm_config_t config = {
        .enable_runtime_tracking = true,
        .enable_static_analysis = true
    };
    msm_context_t* ctx = msm_initialize(&config);

    // Analyze file
    msm_analysis_result_t result = {0};
    if (msm_analyze_file(ctx, "memory.c", &result) == CRRSS_SUCCESS) {
        printf("Total issues: %u\n", result.total_issues);
        printf("Memory leaks: %u\n", result.memory_leaks);
    }

    // Cleanup
    msm_shutdown(ctx);
    return 0;
}
```

Compile and run:

```
gcc -o analyzer analyzer.c -I./tools/crrss -L./tools/crrss/build/lib -lcrrss -lm -lpthread
./analyzer
```

Troubleshooting

Build Issues

Problem: “CRRSS tool not found”

```
# Solution: Build CRRSS
make crrss
# or
cd tools/crrss && make all
```

Problem: “Compiler not found”

```
# Solution: Install GCC or specify compiler
make CC=clang crrss
```

Problem: “C23 standard not supported”

```
# Solution: Update GCC/Clang
sudo apt install gcc-13 # Ubuntu/Debian
# or use newer compiler
make CC=gcc-13 crrss
```

Runtime Issues

Problem: “libcrrss.a not found”

```
# Solution: Build library
cd tools/crrss && make lib
```

Problem: “Permission denied”

```
# Solution: Make executable
chmod +x tools/crrss/build/bin/crrss
```

Problem: “Analysis fails on large files”

```
# Solution: Increase limits
export CRRSS_MAX_THREADS=16
export CRRSS_CACHE_SIZE=5000
```

Hook Issues

See `.git-hooks/README.md` for pre-commit hook troubleshooting.

Performance Tips

1. Use parallel builds:

```
bash
make -j$(nproc) crrss
```


2. Analyze incrementally:

```
bash
# Analyze only changed files
make crrss-analyze FILE=changed_file.c
```

3. Use caching:

```
bash
export CRRSS_CACHE_SIZE=5000
```

4. Limit analysis scope:

```
bash
crrss msm -d moduler_kernel/memory/ --max-issues 50
```

Additional Resources

- **Main README:** `tools/crrss/README.md`
 - **Integration Guide:** `tools/crrss/docs/INTEGRATION.md`
 - **MSM Documentation:** `tools/crrss/msm/README.md`
 - **API Reference:** Header files in respective component directories
 - **Examples:** `tools/crrss/examples/`
 - **Pre-commit Hooks:** `.git-hooks/README.md`
-

Support

For issues, questions, or contributions:

- Open an issue in the BDI repository
 - See main BDI documentation
 - Run `make crrss-help` for quick reference
-

Version Information

```
# Check CRRSS version
crrss --version

# Check build information
make crrss-info
```

CRRSS Usage Guide - Version 1.0.0

Phase 1B Stage 4: Build System Integration & Documentation

Last Updated: October 11, 2025